

NAME : Shri Harshini Devi . A
REG.NO : 192524104
Dept. Name : B.Tech-AIDS
Course Code/Course Name: DSA0602 /Data Handling and Visualization
Date: 07/08/2026

Unit IV Class Activity

1. A university wants to determine whether students who spend more hours studying obtain higher marks.

Dataset

StudyHours Marks

2 45

3 52

4 58

5 63

6 69

7 74

8 81

9 88

Task

- Create the dataset in R.
 - Draw a scatterplot.
 - Add axis labels and title.
 - Interpret whether a positive correlation exists.
2. A hospital records patients' age and systolic blood pressure.
Write a Python program using Matplotlib to:
Create a scatterplot.
Display gridlines.
Change point color to red.
Comment on the observed trend.
 3. A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.
A financial institution has collected information on:
Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

Which variables exhibit the strongest positive correlation?

4. A financial institution has collected information on:

Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

Which variables exhibit the strongest positive correlation?

5. Using the Iris dataset,

Compute the correlation matrix.

Display it as a heatmap using Seaborn.

Identify variables having negative correlation.

6. A healthcare dataset contains:

BMI

Cholesterol

Blood Pressure

Glucose

Heart Rate

Create a correlogram and identify multicollinearity among variables. 7.

A researcher is analyzing 25 features extracted from MRI images. Perform PCA to reduce the dimensionality to two principal components. Plot the transformed data.

8. Using the Iris dataset,

Perform PCA.

Display the summary.

Draw a biplot.

Explain which principal component explains the highest variance.

9. An automobile company has recorded 15 specifications for various cars. Reduce the dimensions using PCA and visualize the first two principal components. 10. A facial recognition dataset contains 128-dimensional facial embeddings. Use t-SNE to visualize the data in two dimensions.

Explain why t-SNE is preferred over PCA for visualization.

11. Using the Iris dataset,

Apply t-SNE and plot the resulting clusters using different colors for each species.

12. A customer segmentation dataset contains 40 behavioral variables. Visualize customer groups using t-SNE.

Discuss whether distinct clusters are formed.

13. A medical image dataset contains 100 extracted features.

Apply UMAP to reduce the dimensions to two.

Visualize the output.

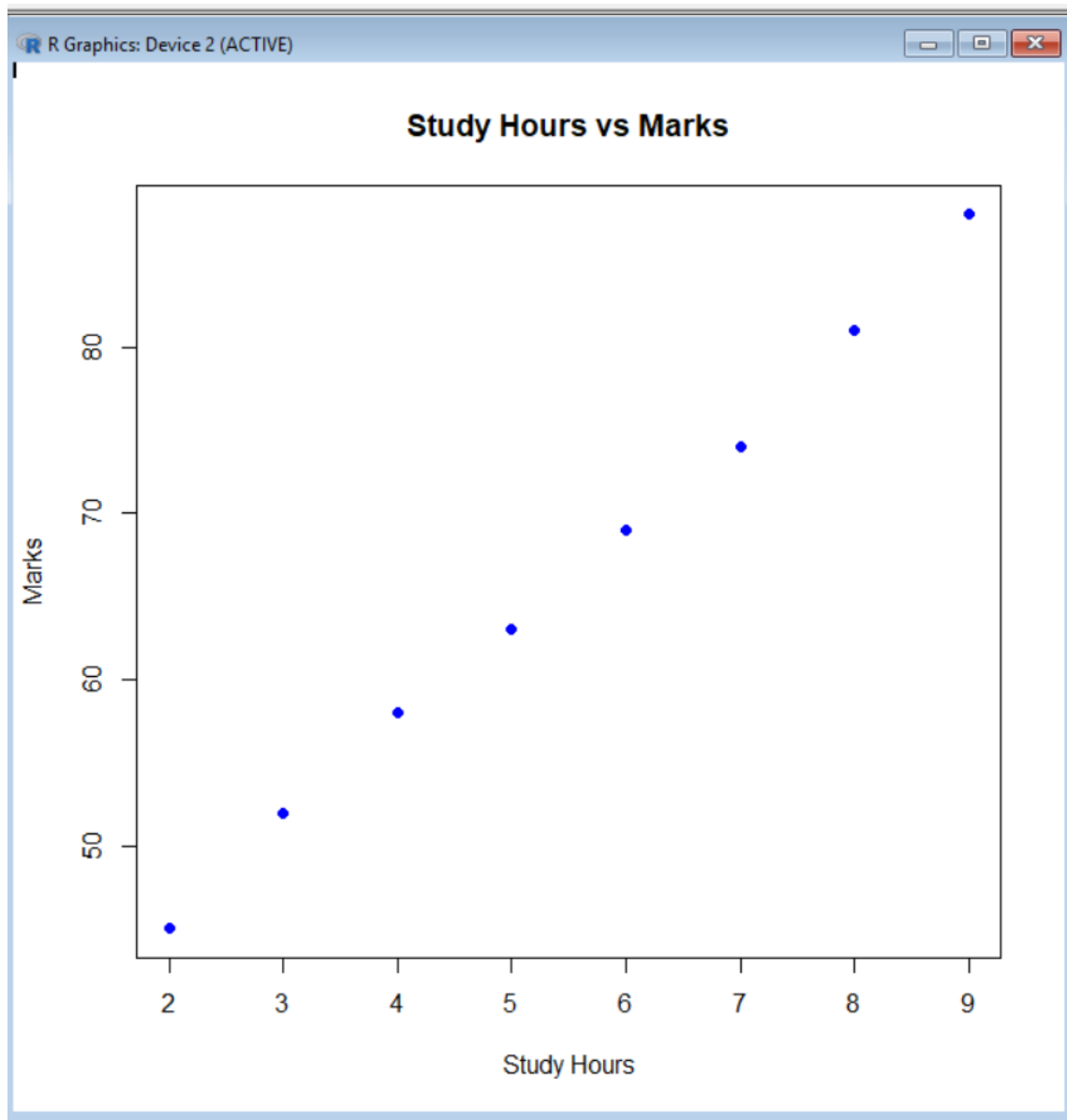
Compare the result with PCA.

14. Apply UMAP to the Iris dataset.
Plot the reduced dimensions using different colors for each species.
15. An e-commerce company has collected customer purchasing behavior consisting of 60 attributes.
Reduce the dimensions using UMAP and identify customer clusters.

```
1. # Create the dataset
StudyHours <- c(2, 3, 4, 5, 6, 7, 8, 9)
Marks <- c(45, 52, 58, 63, 69, 74, 81, 88)

data <- data.frame(StudyHours, Marks)

# Draw scatterplot
plot(data$StudyHours, data$Marks,
      main = "Study Hours vs Marks",
      xlab = "Study Hours",
      ylab = "Marks",
      pch = 19,
      col = "blue")
```



```
2. import matplotlib.pyplot as plt
```

```
# Dataset
```

```
age = [25, 30, 35, 40, 45, 50, 55, 60]
```

```
bp = [110, 115, 118, 122, 128, 135, 140, 148]
```

```
# Scatterplot
```

```
plt.scatter(age, bp, color='red')
```

```
# Labels and Title
```

```
plt.title("Age vs Systolic Blood Pressure")
```

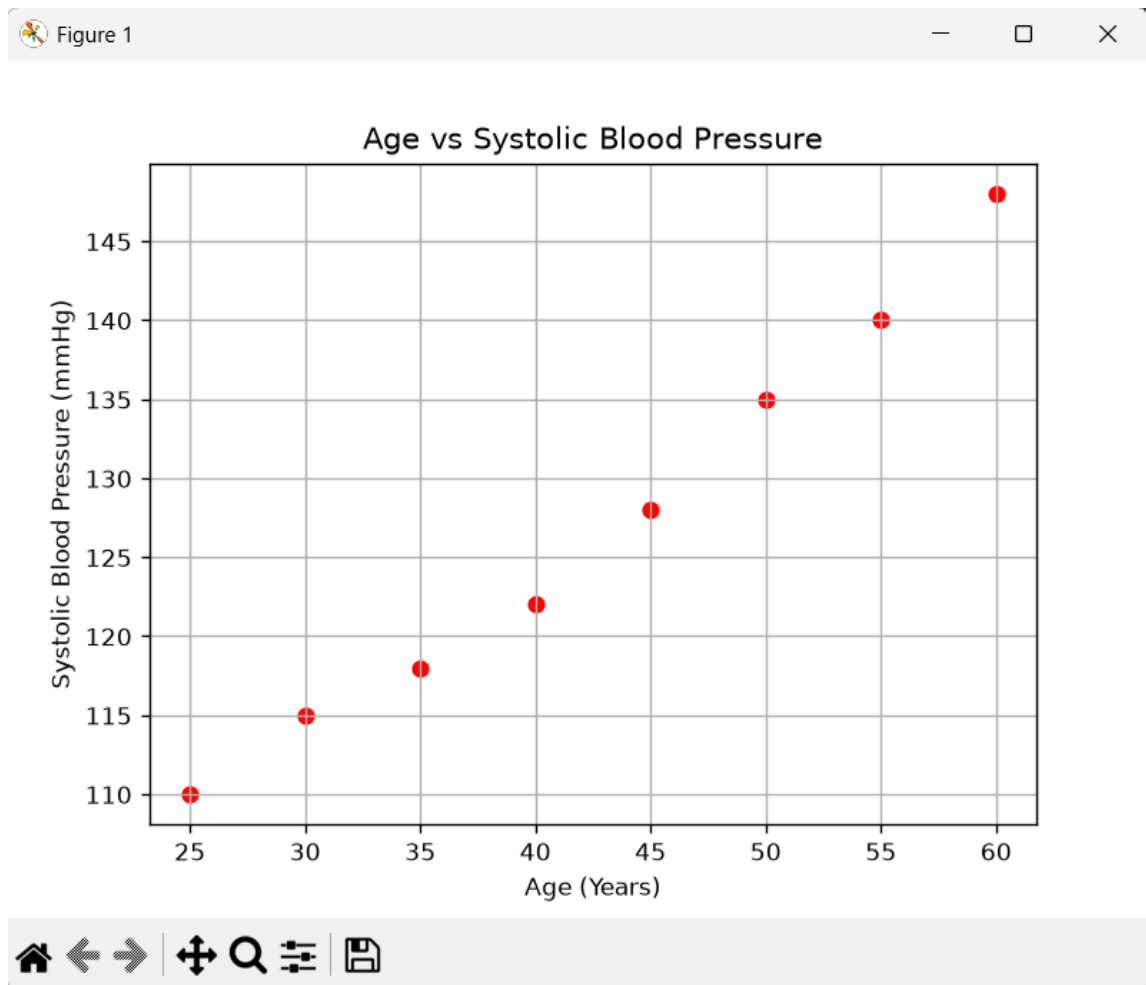
```
plt.xlabel("Age (Years)")
```

```
plt.ylabel("Systolic Blood Pressure (mmHg)")
```

```
# Display gridlines
```

```
plt.grid(True)
```

```
# Show plot  
plt.show()
```



3. (a) To analyze whether **Advertising Expenditure** influences **Monthly Sales** using a scatterplot in **R**.

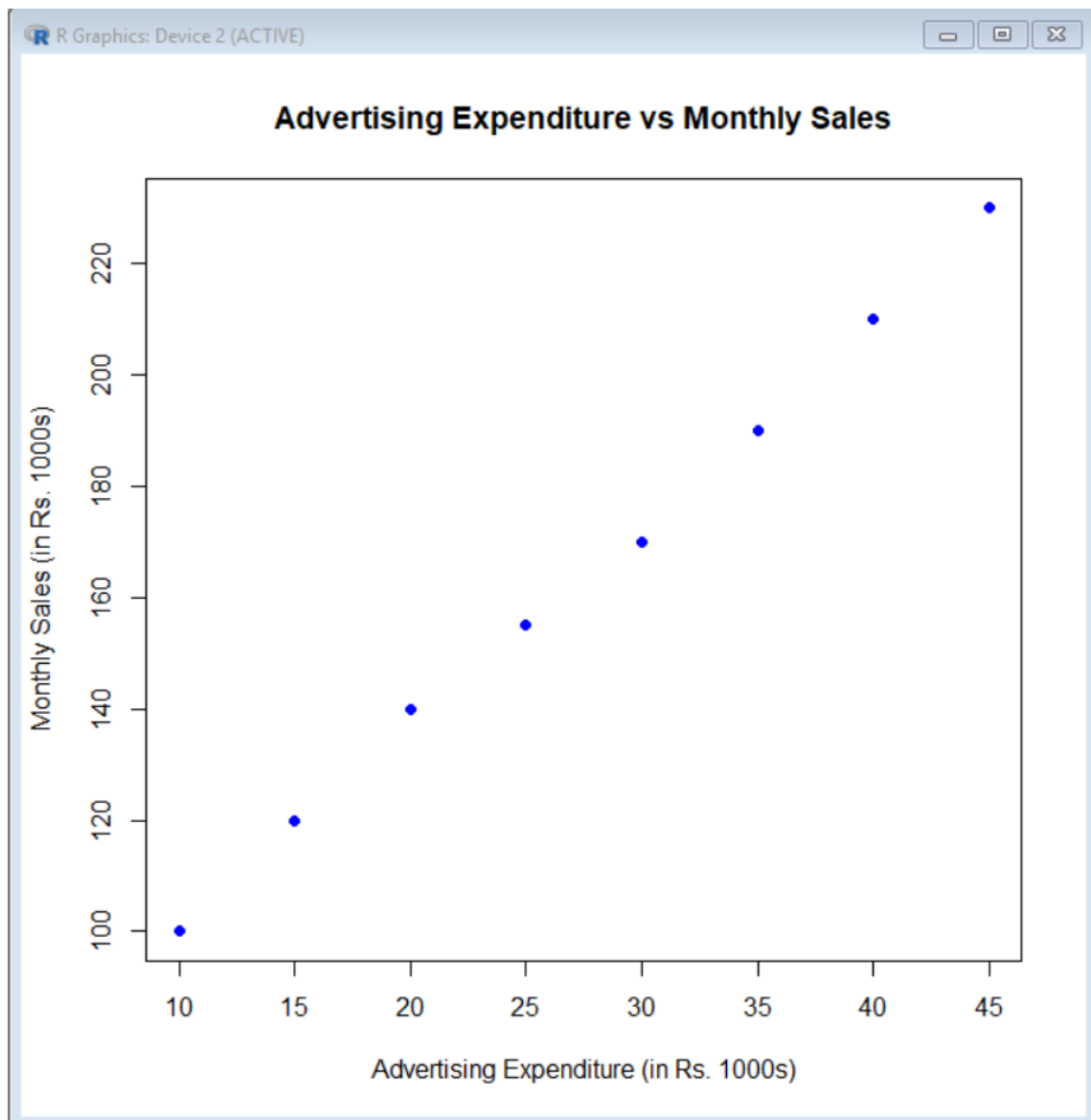
```
# Dataset
```

```
Advertising <- c(10, 15, 20, 25, 30, 35, 40, 45)
```

```
Sales <- c(100, 120, 140, 155, 170, 190, 210, 230)
```

```
# Scatterplot
```

```
plot(Advertising, Sales,  
     main = "Advertising Expenditure vs Monthly Sales",  
     xlab = "Advertising Expenditure (in Rs. 1000s)",  
     ylab = "Monthly Sales (in Rs. 1000s)",  
     pch = 19,  
     col = "blue")
```



3(b) Aim

To generate a **correlation matrix** and **correlogram** for Income, Savings, Loan Amount, and Credit Score in **R**.

R Program

```
# Install package once
```

```
# install.packages("corrplot")
```

```
library(corrplot)
```

```
# Dataset
```

```
data <- data.frame(
```

```
  Income = c(30, 40, 50, 60, 70, 80, 90, 100),
```

```
  Savings = c(5, 8, 12, 15, 18, 22, 26, 30),
```

```
  LoanAmount = c(15, 20, 25, 30, 35, 40, 45, 50),
```

```
  CreditScore = c(600, 620, 650, 680, 710, 740, 770, 800)
```

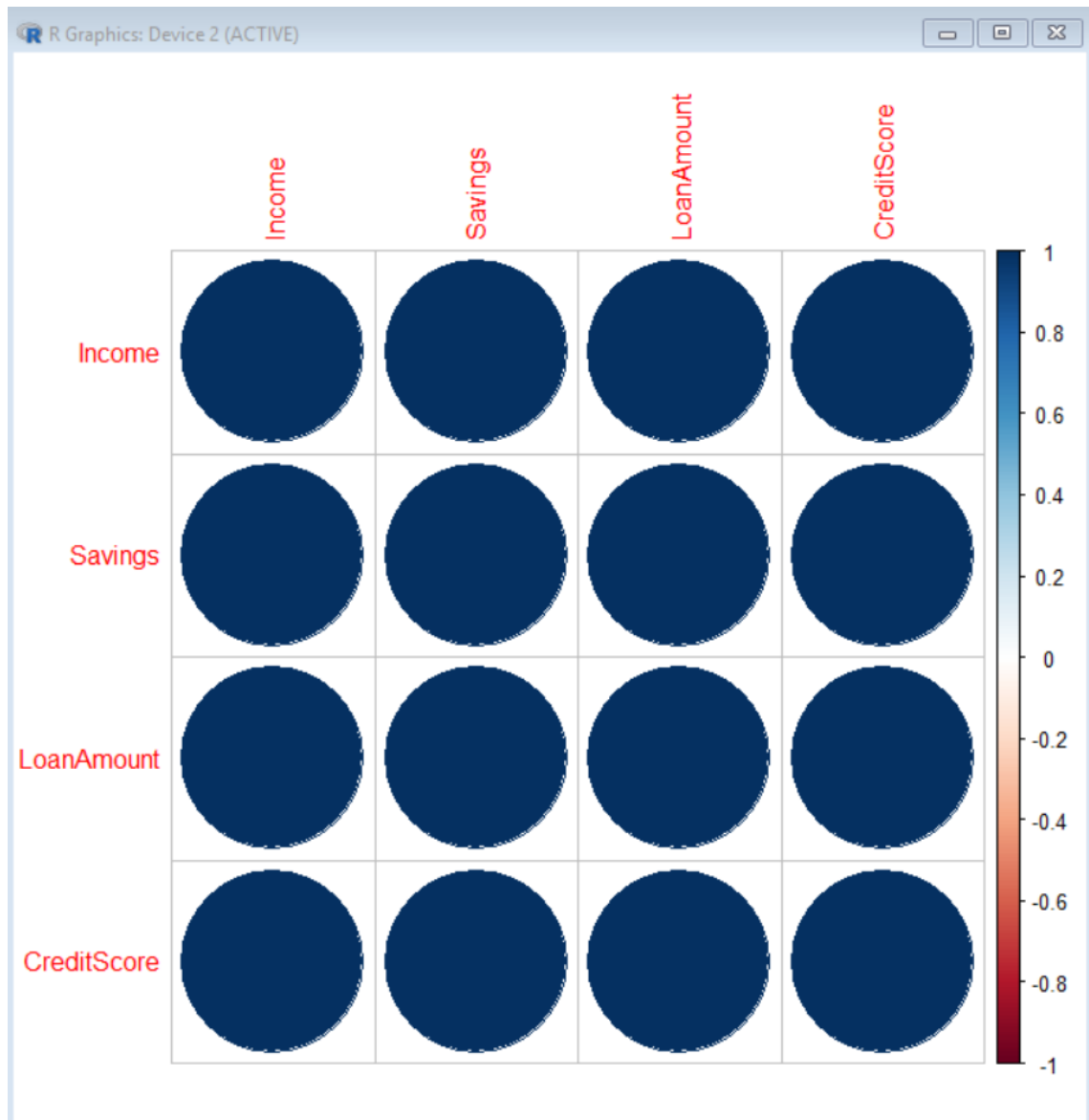
```
)
```

```
# Correlation Matrix
```

```
cor_matrix <- cor(data)
```

```
print(cor_matrix)
```

```
# Correlogram  
corrplot(cor_matrix, method = "circle")
```



```
4. # Install package once  
# install.packages("corrplot")
```

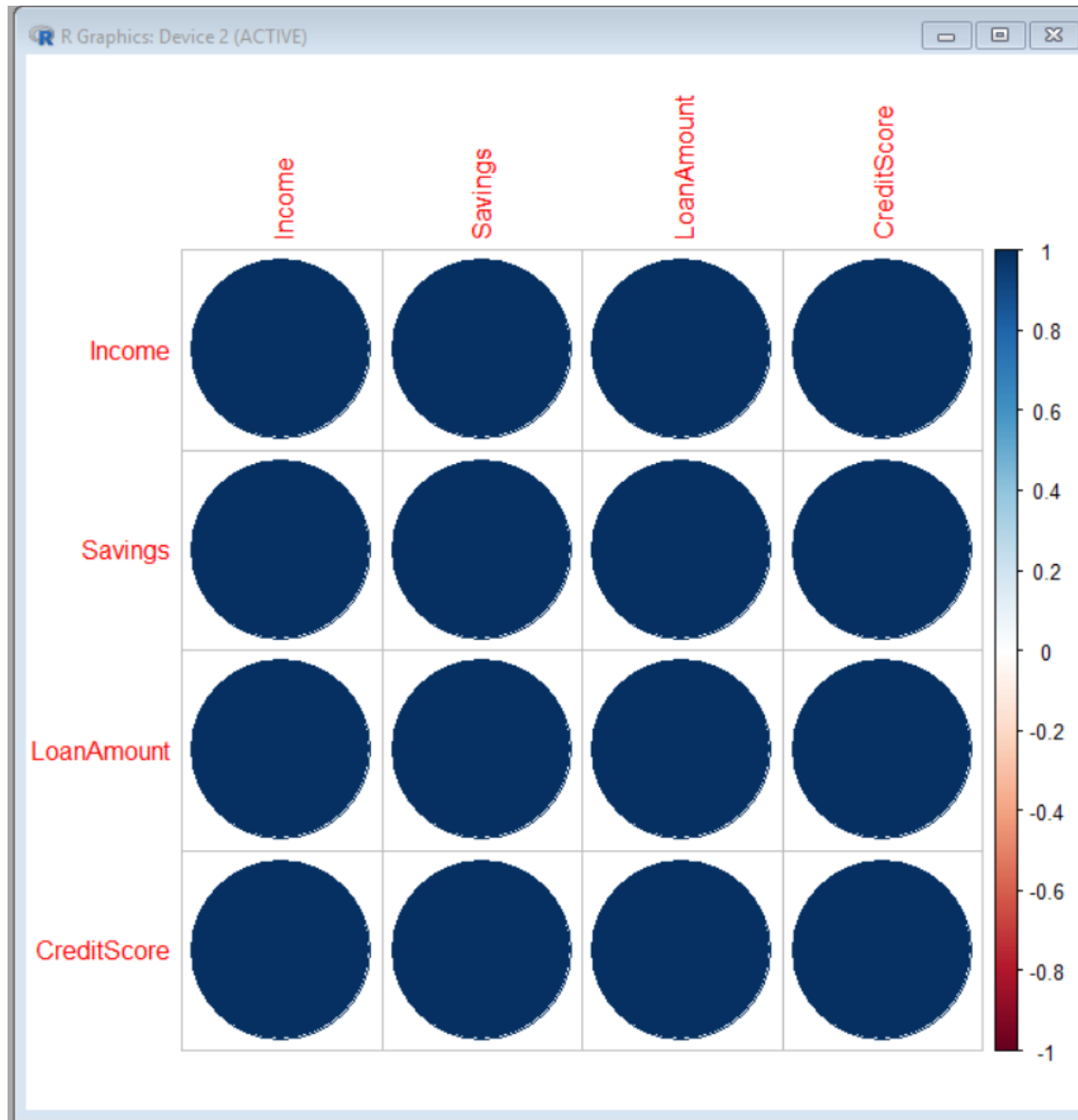
```
library(corrplot)
```

```
# Dataset  
data <- data.frame(  
  Income = c(30, 40, 50, 60, 70, 80, 90, 100),  
  Savings = c(5, 8, 12, 15, 18, 22, 26, 30),  
  LoanAmount = c(15, 20, 25, 30, 35, 40, 45, 50),  
  CreditScore = c(600, 620, 650, 680, 710, 740, 770, 800)
```

)

```
# Correlation Matrix  
cor_matrix <- cor(data)  
print(cor_matrix)
```

```
# Correlogram  
corrplot(cor_matrix, method = "circle")
```



```
5. import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
from sklearn.datasets import load_iris
```

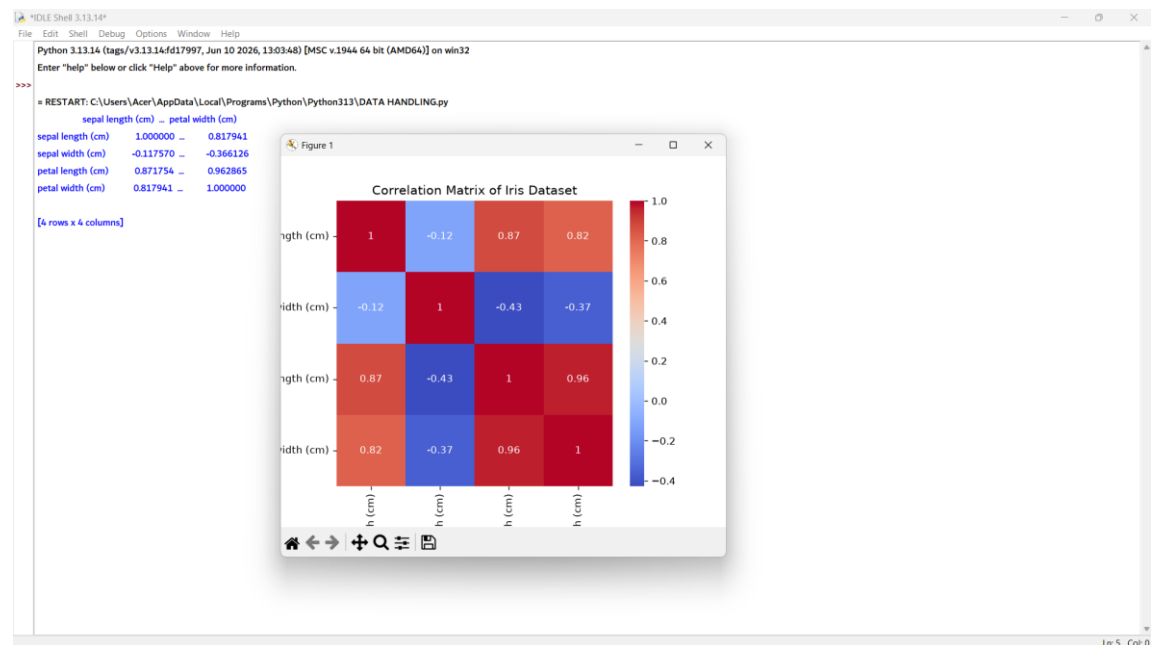
```
# Load Iris dataset
```



```
iris = load_iris(as_frame=True)
df = iris.frame.iloc[:, :-1] # Exclude target column

# Correlation matrix
corr = df.corr()
print(corr)

# Heatmap
plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix of Iris Dataset")
plt.show()
```

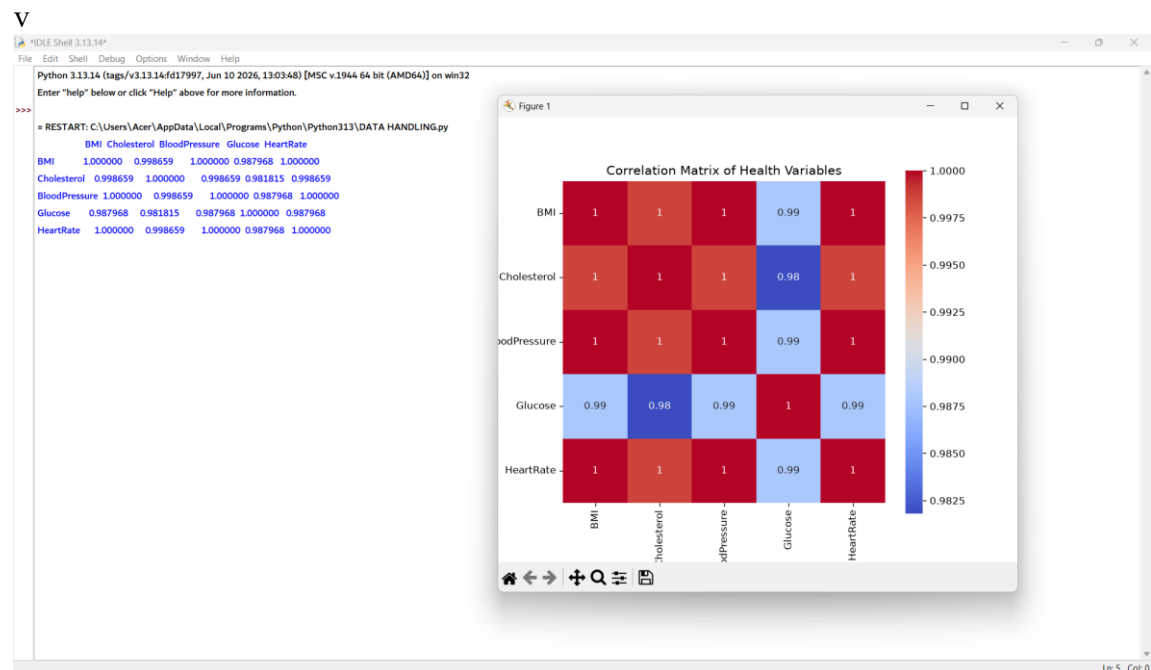


```
6. import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Dataset
df = pd.DataFrame({
    "BMI": [22, 24, 26, 28, 30, 32, 34, 36],
    "Cholesterol": [170, 180, 190, 205, 215, 225, 235, 245],
    "BloodPressure": [110, 115, 120, 125, 130, 135, 140, 145],
    "Glucose": [85, 90, 95, 100, 110, 120, 130, 140],
    "HeartRate": [68, 70, 72, 74, 76, 78, 80, 82]
})

# Correlation Matrix
corr = df.corr()
print(corr)
```

```
# Correlogram (Heatmap)
plt.figure(figsize=(7,6))
sns.heatmap(corr, annot=True, cmap="coolwarm", square=True)
plt.title("Correlation Matrix of Health Variables")
plt.show()
```

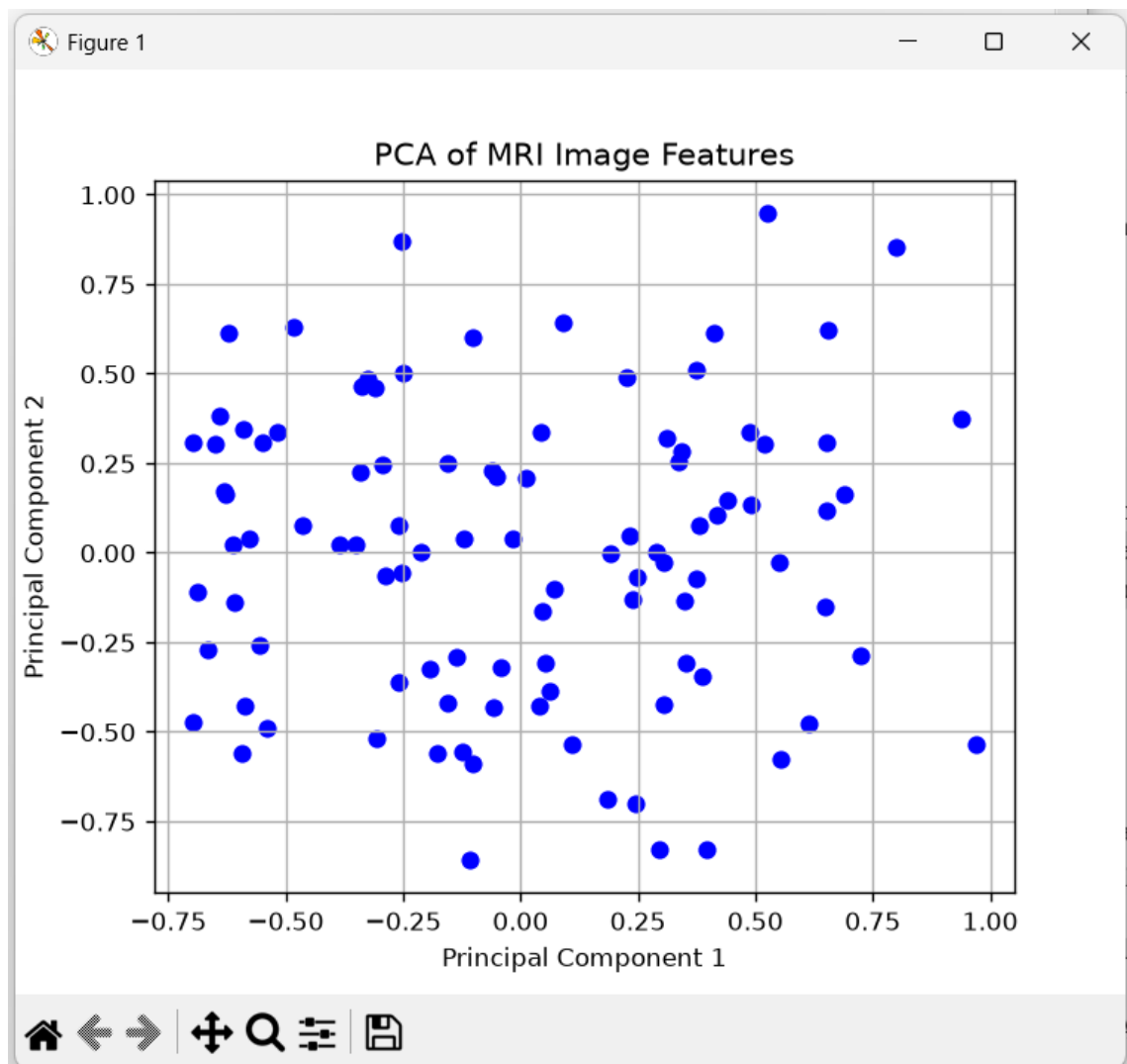


```
7. import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

```
# Generate sample dataset (100 samples, 25 features)
np.random.seed(42)
X = np.random.rand(100, 25)
```

```
# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

```
# Plot the transformed data
plt.figure(figsize=(6,5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], color='blue')
plt.title("PCA of MRI Image Features")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.show()
```



```
8. # Load Iris dataset
data(iris)

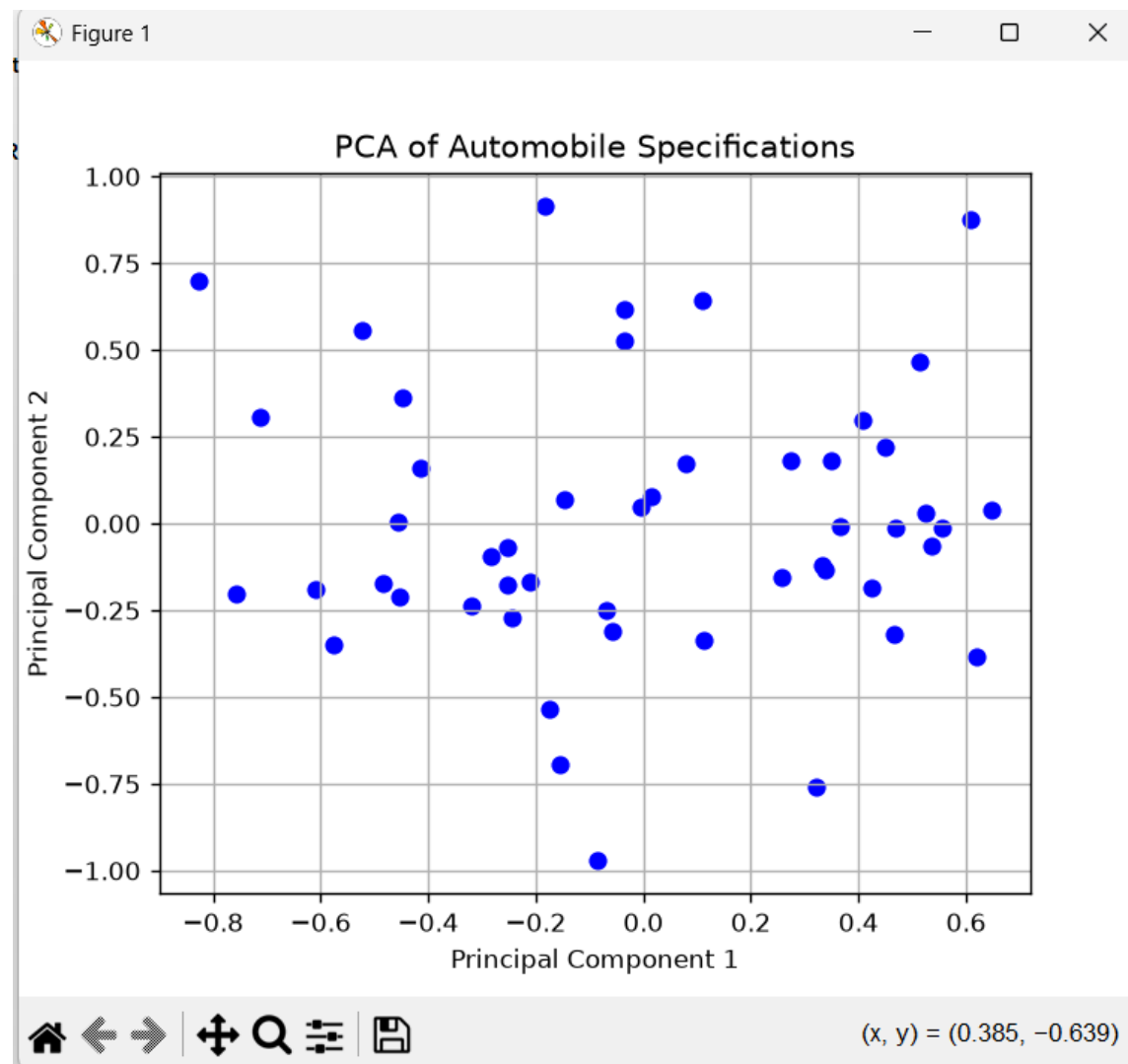
# Perform PCA
pca_result <- prcomp(iris[,1:4], scale. = TRUE)

# Display summary
summary(pca_result)

# Draw biplot
biplot(pca_result,
       main = "PCA Biplot of Iris Dataset")
```



```
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.show()
```

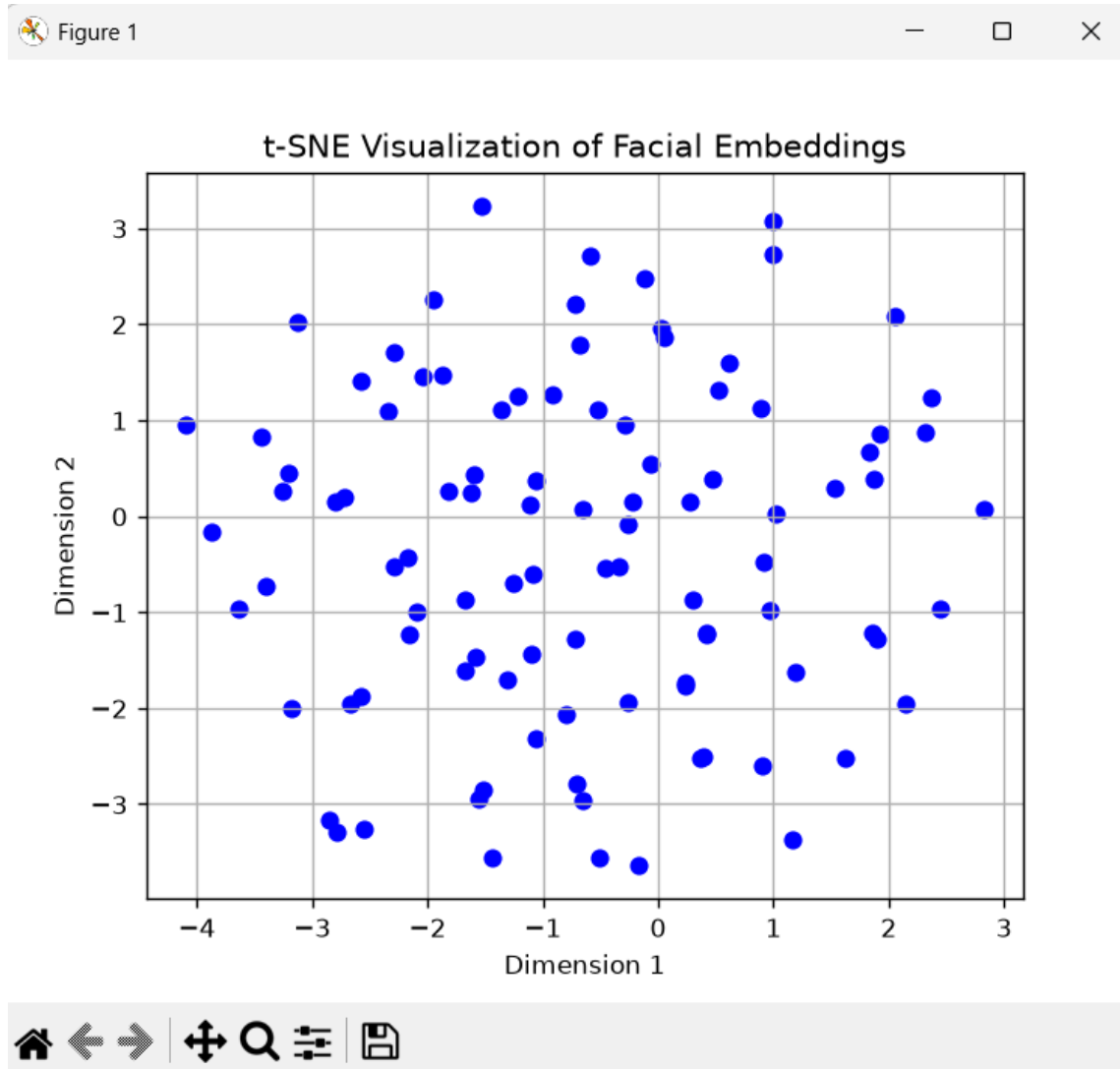


```
10. import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# Generate sample dataset (100 samples, 128 features)
np.random.seed(42)
X = np.random.rand(100, 128)

# Apply t-SNE
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)
```

```
# Plot the transformed data
plt.figure(figsize=(6,5))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], color='blue')
plt.title("t-SNE Visualization of Facial Embeddings")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.grid(True)
plt.show()
```



```
11. import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
```

```
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
```

```
labels = iris.target_names

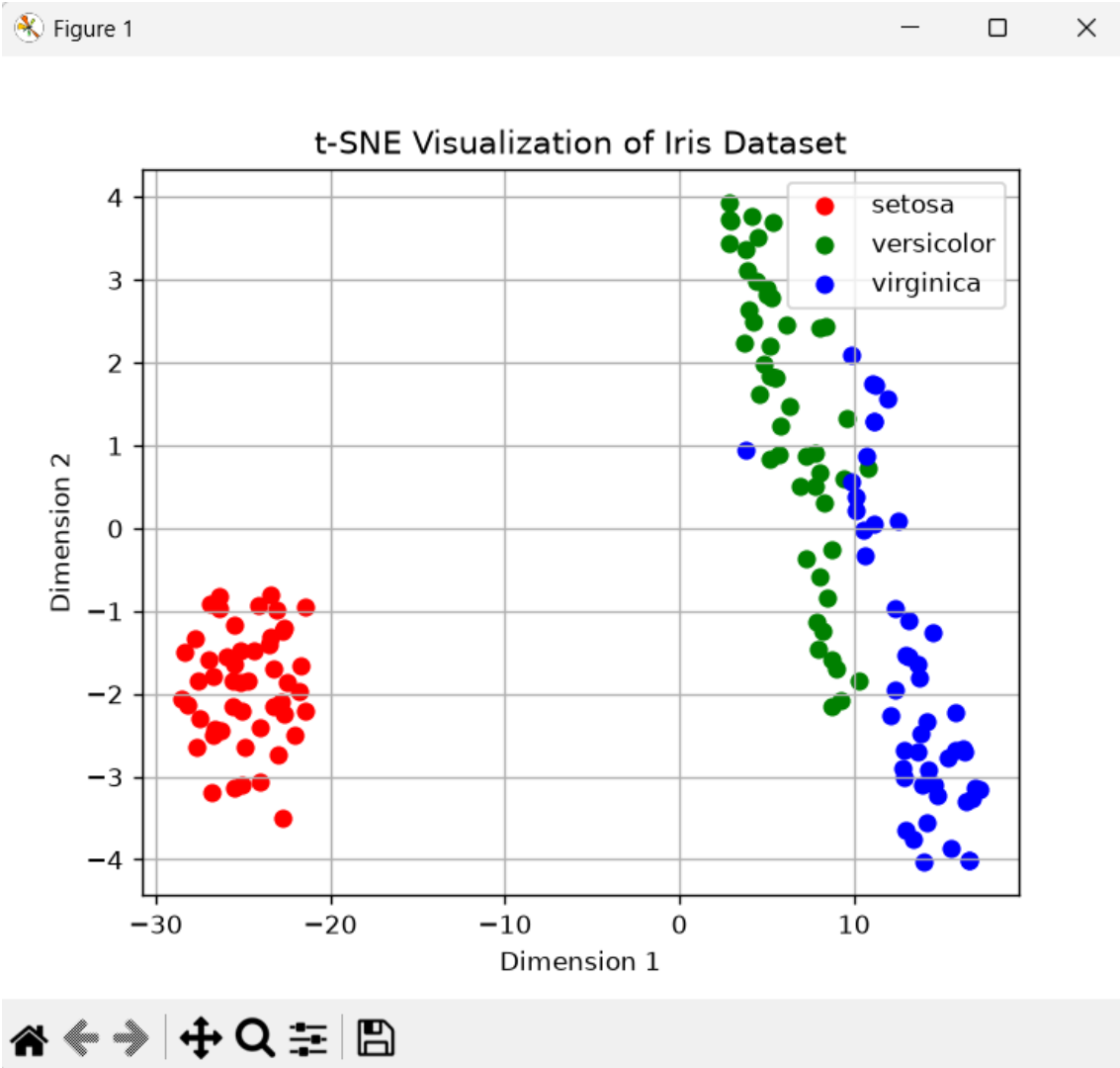
# Apply t-SNE
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

# Plot clusters
plt.figure(figsize=(6,5))

colors = ['red', 'green', 'blue']

for i in range(3):
    plt.scatter(X_tsne[y == i, 0],
                X_tsne[y == i, 1],
                color=colors[i],
                label=labels[i])

plt.title("t-SNE Visualization of Iris Dataset")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.legend()
plt.grid(True)
plt.show()
```



```
12. import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

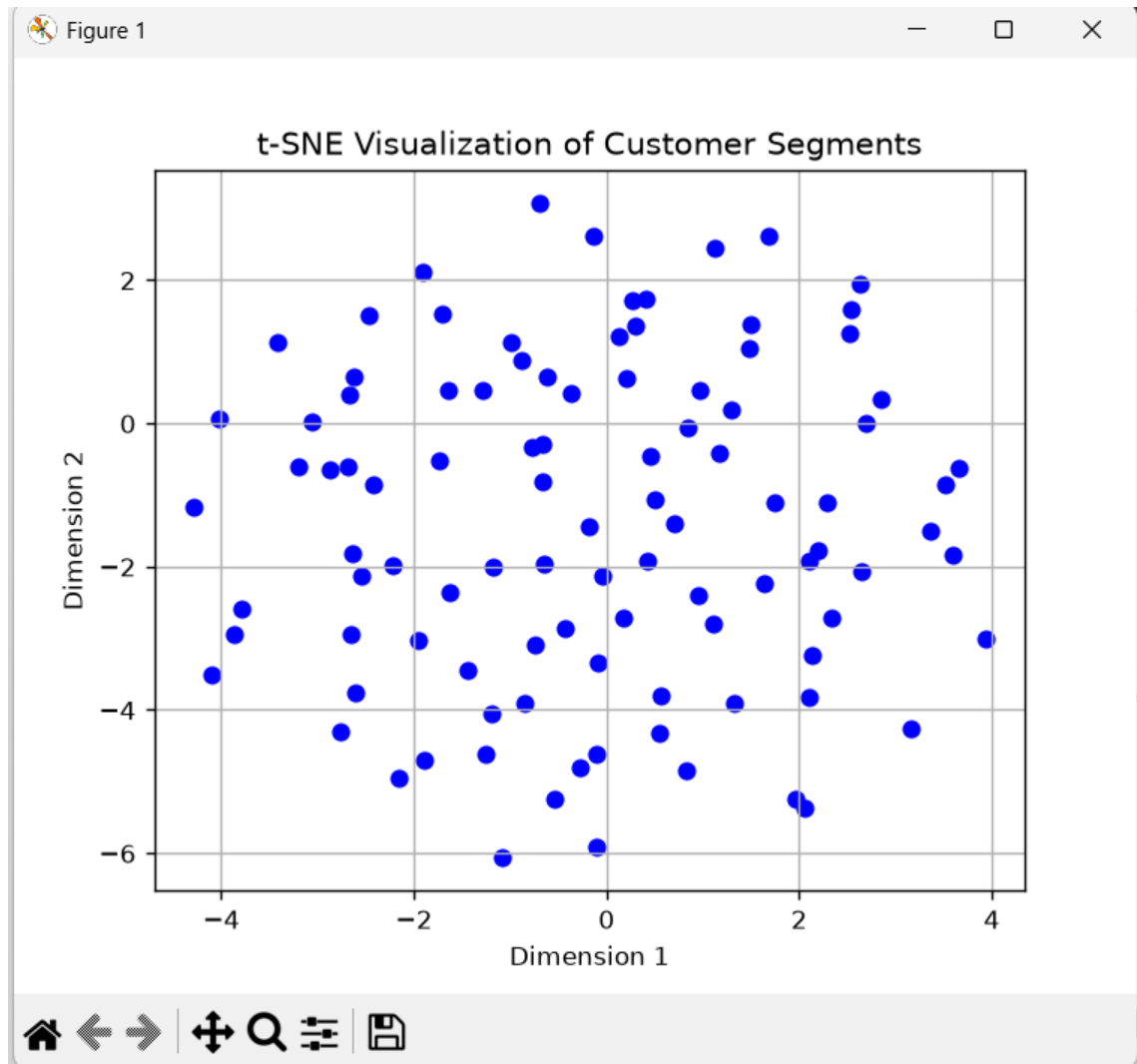
# Generate sample dataset (100 customers, 40 behavioral variables)
np.random.seed(42)
X = np.random.rand(100, 40)

# Apply t-SNE
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

# Plot the transformed data
plt.figure(figsize=(6,5))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], color='blue')
plt.title("t-SNE Visualization of Customer Segments")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
```



```
plt.grid(True)
plt.show()
```



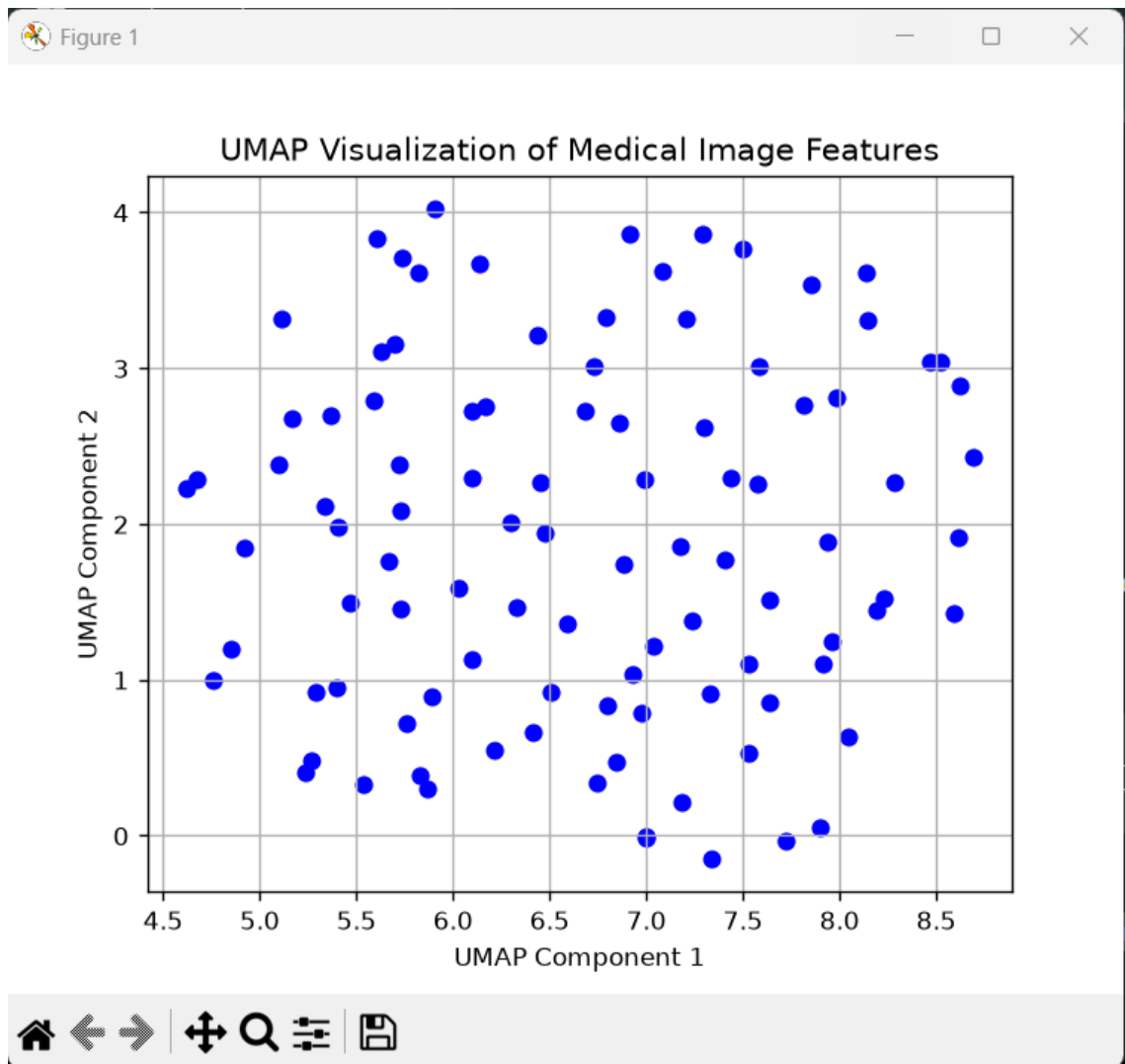
```
13. import numpy as np
import matplotlib.pyplot as plt
import umap

# Generate sample dataset (100 samples, 100 features)
np.random.seed(42)
X = np.random.rand(100, 100)

# Apply UMAP
reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X)

# Plot UMAP output
plt.figure(figsize=(6,5))
plt.scatter(X_umap[:, 0], X_umap[:, 1], color='blue')
```

```
plt.title("UMAP Visualization of Medical Image Features")
plt.xlabel("UMAP Component 1")
plt.ylabel("UMAP Component 2")
plt.grid(True)
plt.show()
```



```
14. import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import umap
```

```
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
labels = iris.target_names
```

```
# Apply UMAP
```

```
reducer = umap.UMAP(n_components=2, random_state=42)
```

```
X_umap = reducer.fit_transform(X)
```

```
# Plot the reduced dimensions
```

```
plt.figure(figsize=(6,5))
```

```
colors = ['red', 'green', 'blue']
```

```
for i in range(3):
```

```
    plt.scatter(X_umap[y == i, 0],
```

```
                X_umap[y == i, 1],
```

```
                color=colors[i],
```

```
                label=labels[i])
```

```
plt.title("UMAP Visualization of Iris Dataset")
```

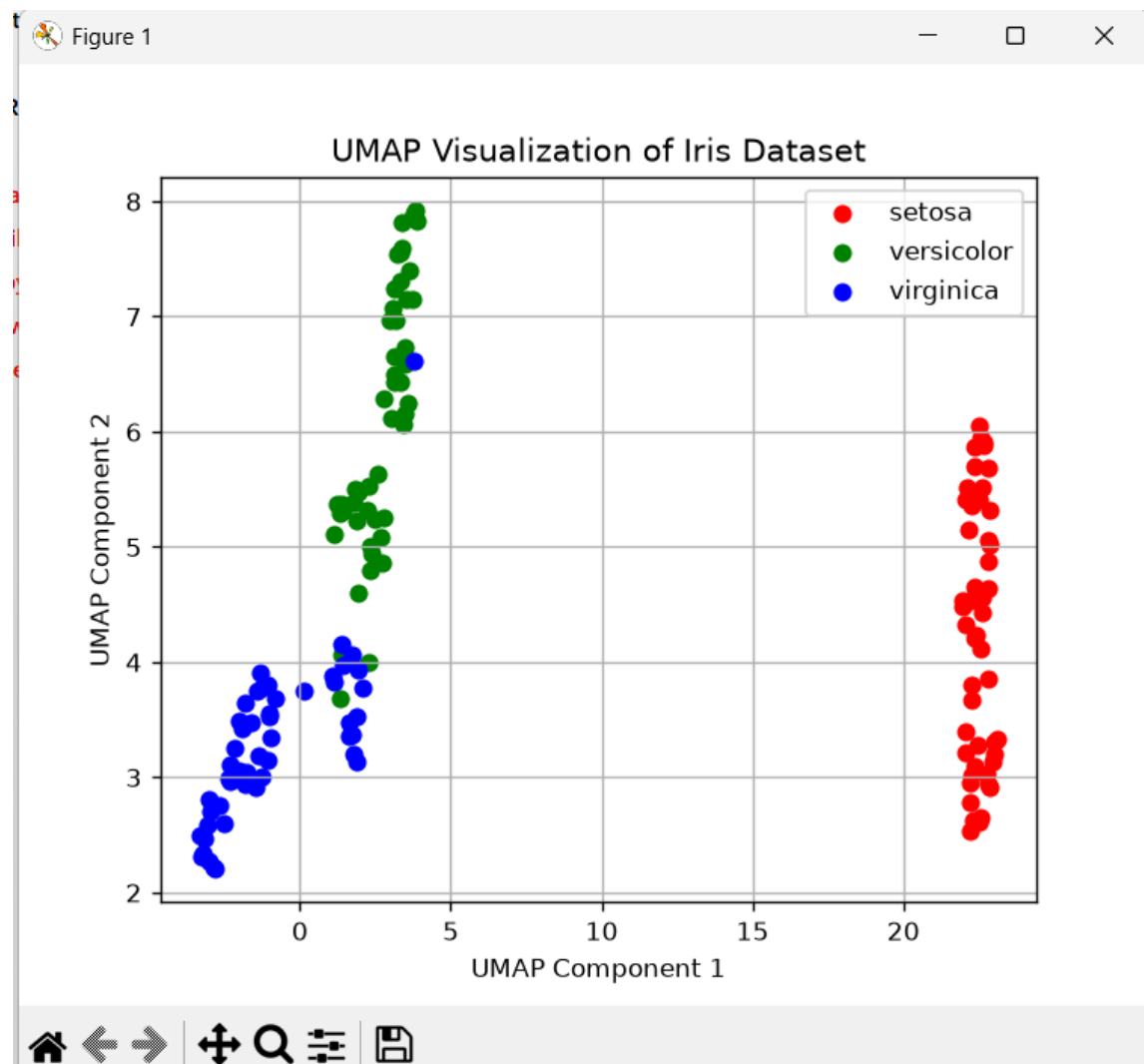
```
plt.xlabel("UMAP Component 1")
```

```
plt.ylabel("UMAP Component 2")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



```
15. import numpy as np
import matplotlib.pyplot as plt
import umap
from sklearn.cluster import KMeans

# Generate sample dataset (200 customers, 60 attributes)
np.random.seed(42)
X = np.random.rand(200, 60)

# Apply UMAP
reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X)

# Apply K-Means clustering
kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X_umap)

# Plot clusters
plt.figure(figsize=(7, 5))
plt.scatter(X_umap[:, 0], X_umap[:, 1],
            c=clusters, cmap='viridis')

plt.title("UMAP Visualization of Customer Clusters")
plt.xlabel("UMAP Component 1")
plt.ylabel("UMAP Component 2")
plt.grid(True)
plt.show()
```

